

MANUAL TÉCNICO — AGENTE MCKENNA GROUP

> Nivel: desarrollador junior que ve el código por primera vez.

> Objetivo: entender qué hace cada archivo, cada función, y por qué está escrito así.

****Manual de usuario (PDF operativo):**** se genera con ``source venv/bin/activate && python3 scripts/generar_manual.py`` (opcional ``--enviar`` para adjuntar por correo). Ese PDF cubre comandos del grupo, flujos de negocio y glosario; este ``MANUAL.md`` es la referencia técnica del código.

ÍNDICE

- 0. [Actualizar repositorio (Git)](#0-actualizar-repositorio-git)
- 1. [agente_pro.py](#1-agente_propy)
- 2. [webhook_meli.py](#2-webhook_melipy)
- 3. [app/core.py](#3-appcorepy)
- 4. [app/routes.py](#4-approutespy)
- 5. [app/cli.py](#5-appclipy)
- 6. [app/services/meli.py](#6-appservicesmelipy)
- 7. [app/services/siigo.py](#7-appservicessiigopy)
- 8. [app/tools/system_tools.py](#8-apptoolssystem_toolspy)
- 9. [APIs de Generación Multimedia (consola)](#9-apis-de-generación-de-contenido-multimedia-consola)
- 10. [Nuevas Skills (Herramientas Autónomas)](#10-nuevas-skills-herramientas-autónomas)
- 11. [Observabilidad, auditoría de scripts, backup y Git](#11-observabilidad-auditoría-de-scripts-backup-y-git)
- 12. [Actualizaciones operativas (2026-04-23)](#12-actualizaciones-operativas-2026-04-23)

0. Actualizar repositorio (Git)

Flujo recomendado para actualizar sin perder trabajo local:

```
git status --short --branch
git branch -vv
git fetch origin
git pull --rebase origin main
```

Si hay archivos modificados en local, primero hacer commit o ``git stash`` para evitar conflictos.

Si ``main`` no es rama remota por defecto, usar la rama que salga en ``git branch -vv`` (por ejemplo ``master``).

1. ``agente_pro.py``

¿Para qué sirve?

Este es el **archivo de entrada principal** del sistema. Piénsalo como el interruptor maestro de luz: no hace el trabajo pesado, solo enciende todo lo demás en el orden correcto.

Su trabajo es:

1. Cargar las contraseñas y claves desde el archivo `.env``
2. Crear la aplicación web (Flask)
3. Conectar el cerebro de IA al servidor
4. Lanzar el menú de consola **en paralelo** con el servidor web
5. Encender el servidor web en el puerto 8081

Analogía: Es como el gerente de apertura de un restaurante. No cocina, no atiende mesas, pero llega primero, abre la puerta, enciende las luces y llama a cada persona a su puesto.

Explicación línea por línea

```
import os
import threading
import logging
from flask import Flask
from dotenv import load_dotenv
```

- ``os`` — para leer variables del sistema operativo
- ``threading`` — para ejecutar dos cosas al mismo tiempo (el servidor web y el menú de consola)
- ``logging`` — para controlar qué mensajes se imprimen en pantalla
- ``Flask`` — el framework que convierte este programa en un servidor web que puede recibir peticiones HTTP
- ``load_dotenv`` — función que lee el archivo `.env`` y carga todas las variables (como ``GOOGLE_API_KEY``) al entorno del proceso

```
load_dotenv()
```

Esta línea se ejecuta apenas arranca el programa. Lee el archivo `.env`` y hace que todas las claves API queden disponibles con ``os.getenv("NOMBRE_CLAVE")`` en cualquier parte del código. Si esta línea no existiera, todas las claves serían ``None`` y nada funcionaría.

```
from app.routes import register_routes
from app.cli import iniciar_cli
from app.core import configurar_ia
```

Importa tres funciones de otros módulos:

- ``register_routes`` — registra las URLs que el servidor web va a escuchar (ej: ``/whatsapp``)
- ``iniciar_cli`` — arranca el menú interactivo de la consola
- ``configurar_ia`` — inicializa el cliente **Anthropic Claude** y registra todas las herramientas del agente (WhatsApp, ``/chat``, CLI)

```
def create_app():
```

Esta es una **fábrica de aplicaciones** (Application Factory). En lugar de crear ``app = Flask(__name__)`` directamente en el módulo global, se hace dentro de una función. ¿Por qué? Porque así es más fácil de probar y de configurar sin efectos secundarios al importar el módulo.

```
app = Flask(__name__)
```

Crea la instancia de Flask. `\_\_name\_\_` le dice a Flask el nombre del módulo actual, para que sepa dónde buscar archivos de configuración y plantillas.

```
log = logging.getLogger('werkzeug')
log.setLevel(logging.ERROR)
```

`werkzeug` es la librería interna de Flask que imprime en consola cada petición HTTP que llega (por ejemplo: `POST /whatsapp 200`). Aquí se silencia para que esos mensajes no ensucien el menú de consola con el que interactúa el operador. Solo se mostrarán errores reales.

```
configurar_ia(app)
```

Llama a la función de `app/core.py` que conecta **Claude** (`ANTHROPIC\_API\_KEY`) y registra todas las herramientas disponibles. **Gemini** se usa en otro flujo (preventa MeLi con ficha, `app/services/meli\_preventa.py`), no aquí. Se pasa `app` como parámetro porque en el futuro podría necesitarse la instancia de Flask para guardar el modelo en `app.config`.

```
register_routes(app)
```

Le dice a Flask qué URLs escuchar y qué función ejecutar cuando llegue cada petición. Esto se define en `app/routes.py`.

```
if __name__ == "__main__":
```

Esta condición significa: "solo ejecuta este bloque si alguien corre este archivo directamente con `python3 agente\_pro.py`". Si otro archivo importa `agente\_pro`, este bloque no se ejecuta.

```
app = create_app()
print("■ Iniciando el Agente de McKenna Group...")
```

Crea la aplicación usando la fábrica y muestra un mensaje de bienvenida.

```
cli_thread = threading.Thread(target=iniciar_cli, daemon=True)
cli_thread.start()
```

Aquí está el truco del paralelismo. `threading.Thread` crea un hilo secundario que ejecuta `iniciar\_cli` al mismo tiempo que el hilo principal corre el servidor web. `daemon=True` significa que si el programa principal se cierra, este hilo también muere automáticamente (no queda colgado en memoria).

```
app.run(host='0.0.0.0', port=8081, debug=False)
```

Arranca el servidor web Flask:

- `host='0.0.0.0'` — escucha peticiones de cualquier IP de la red (no solo `localhost`)
- `port=8081` — usa el puerto 8081
- `debug=False` — modo producción: no recarga automáticamente el servidor al guardar un archivo

Conexiones con otros archivos

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `app/core.py` → `configurar\_ia()` | Inicializar Claude y herramientas del agente |

| `app/routes.py` → `register\_routes()` | Registrar URLs del servidor |

| `app/cli.py` → `iniciar\_cli()` | Lanzar el menú de consola |

| `.env` (vía `load\_dotenv`) | Leer claves API y configuración |

¿Por qué se hizo así?

****Patrón Application Factory:**** En proyectos Flask profesionales, no se crea ``app`` como variable global al importar el módulo. Se usa una función ``create_app()`` porque permite crear múltiples instancias de la app (útil para testing) y evita problemas de importación circular.

****Hilos (threading):**** El menú de consola necesita ``input()`` que bloquea el hilo. Si corriera en el mismo hilo que el servidor, el servidor dejaría de responder peticiones mientras espera que el usuario escriba. Al poner el CLI en un hilo separado, ambos funcionan independientemente.

2. ``webhook_meli.py``

¿Para qué sirve?

Este archivo es un ****servidor web paralelo y autónomo**** diseñado específicamente para recibir preguntas de clientes que llegan desde las publicaciones de Mercado Libre. Cuando alguien hace una pregunta en una publicación del vendedor, MeLi envía una notificación a este servidor, que la procesa con IA y publica la respuesta automáticamente.

****Analogía:**** Imagina que tienes un empleado dedicado exclusivamente a revisar el buzón de preguntas de MeLi, leer cada pregunta, consultarle al gerente (la IA) qué responder, y escribir la respuesta. Este archivo es ese empleado.

****Nota importante:**** Este archivo corre en el puerto ****8080****, separado del servidor principal que corre en el ****8081****. Son dos servidores distintos.

Explicación línea por línea

```
import os
import requests
import threading
import time
from flask import Flask, request, jsonify
from dotenv import load_dotenv
from app.core import obtener_respuesta_ia, configurar_ia
from app.utils import enviar_whatsapp_reporte, refrescar_token_meli, ...
from preventa_meli import procesar_nueva_pregunta
from app.sync import sincronizar_stock_todas_las_plataformas
```

Importaciones clave:

- ``configurar_ia`` — inicializa ****Claude**** en este proceso (necesario para ``/chat`` en :8080).
- ``procesar_nueva_pregunta`` — ****preventa MeLi****: ficha en Sheets + ****Gemini**** en ``meli_preventa.py``; no usa el bucle de herramientas de WhatsApp.
- ``sincronizar_stock_todas_las_plataformas`` — tras ventas MeLi, alinea stock hacia la ****página web**** (API) y MeLi.
- ``refrescar_token_meli`` — token válido para la API de MeLi.
- ``enviar_whatsapp_reporte`` — alertas al equipo (grupos configurados en ``.env``).

```
app = Flask(__name__)
```

Aquí se crea ``app`` directamente como variable global, a diferencia de ``agente_pro.py`` que usa una fábrica. Esto es más simple pero menos flexible (no se puede usar en tests fácilmente).

Función `obtener\_nombre\_producto(item\_id)`

```
def obtener_nombre_producto(item_id):  
    """Obtiene el título de la publicación de Mercado Libre."""  
    token_actual = refrescar_token_meli() or os.environ.get("MELI_ACCESS_TOKEN")
```

Llama a `refrescar\_token\_meli()` para obtener un token válido. Si esa función falla y retorna `None`, usa `or` para intentar obtener el token directamente de la variable de entorno como respaldo.

```
url = f"https://api.mercadolibre.com/items/{item_id}"  
headers = {"Authorization": f"Bearer {token_actual}"}  
try:  
    res = requests.get(url, headers=headers)  
    if res.status_code == 200:  
        return res.json().get('title', 'Producto desconocido')  
except Exception as e:  
    print(f"Error obteniendo nombre del producto: {e}")  
    return "Producto desconocido"
```

Consulta la API de MeLi para obtener el título de la publicación (ej: "Ácido Cítrico 500g"). Si algo falla, retorna `"Producto desconocido"` en lugar de romper el programa. Esto se llama ****manejo defensivo de errores****.

Función `responder\_en\_mercado\_libre(question\_id, texto)`

```
def responder_en_mercado_libre(question_id, texto):  
    """Envía la respuesta final a la API de Mercado Libre."""  
    token_actual = refrescar_token_meli() or os.environ.get("MELI_ACCESS_TOKEN")  
    url = "https://api.mercadolibre.com/answers"  
    headers = {  
        "Authorization": f"Bearer {token_actual}",  
        "Content-Type": "application/json"  
    }  
    data = {"question_id": question_id, "text": texto}  
    try:  
        response = requests.post(url, json=data, headers=headers)  
        return response.status_code  
    except Exception as e:  
        print(f"Error al responder en MeLi: {e}")  
        return 500
```

Publica la respuesta generada por la IA en la publicación de MeLi. El endpoint `/answers` de MeLi recibe el ID de la pregunta y el texto de la respuesta. Retorna el código HTTP de la respuesta (200/201 = éxito, 500 = error).

Preventa, órdenes y mensajes (`POST /notifications`)

****Producción:**** en Mercado Libre, la ***Callback URL*** de notificaciones debe ser ****`https://bot.mckennagroup.co/notifications`**** (mismo path que expone `webhook\_meli.py` en el puerto ****8080****; el túnel `bot.mckennagroup.co` debe apuntar a ese proceso).

El cuerpo JSON trae `topic` y `resource`:

| `topic` | Acción en hilo aparte |

|-----|-----|

| `questions` | `procesar\_nueva\_pregunta(question\_id)` — flujo preventa (Sheets + Gemini + cola JSON).

Deduplicación ~5 min. |

| `orders\_v2` | `procesar\_orden\_meli(order\_id)` — lee ítems, SKU y `available\_quantity` en MeLi y llama `sincronizar\_stock\_todas\_las\_plataformas` hacia web + MeLi. |

| `messages` | Posventa MeLi: notificación al grupo según mensajes del comprador. |

Posventa (`procesar\_mensaje\_posventa` en `webhook\_meli.py` y duplicado legado en `app/routes.py`): las llamadas GET a la API de mensajes de MeLi llevan cabecera `x-version: 2`. El `resource` suele ser ruta `/messages/packs/{pack_id}/...`; en `app/routes.py`, si viene como `orders/{id}`, ese id se usa como pack para listar mensajes. En `webhook_meli.py`, si no hay `packs/` en la ruta, se aplican estrategias de resolución (mensaje directo, metadatos, búsqueda de órdenes recientes). El ID del mensaje para deduplicar sale de `id` o `message_id` (`meli_postventa_id_mensaje` en `app/utlis.py`). El texto mostrado en WhatsApp usa `meli_postventa_texto_para_notif`: normaliza `text` (string u objeto con `plain`) y, si solo hay `adjuntos` (p. ej. PDF de RUT), arma un resumen para que el equipo abra MeLi. `enviar_whatsapp_reporte` reintenta ante `503` del puente Node (WhatsApp aún sincronizando) y ante errores de conexión transitorios. Tras preventa automática, si el reporte a WhatsApp falla, `preventa_meli.py` registra en consola el fallo (la respuesta en MeLi puede haberse publicado igual).

Regla crítica: responder `200 OK` al instante; todo lo pesado va en `threading.Thread`.

```
@app.route("/notifications", methods=["POST"])
def notifications():
    data = request.get_json()
    topic = data.get("topic") if data else None
    if topic == "questions":
        question_id = data.get("resource", "").split("/)[-1]
        # dedup + Thread(target=procesar_nueva_pregunta, args=(question_id,))
    elif topic == "orders_v2":
        order_id = data.get("resource", "").split("/)[-1]
        # Thread(target=_procesar_orden_meli, args=(order_id,))
    # ...
    return jsonify({"status": "ok"}), 200
```

Conexiones con otros archivos

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `preventa_meli.py` → `procesar_nueva_pregunta()` | Preventa con ficha + Gemini |

| `app/sync.py` → `sincronizar_stock_todas_las_plataformas()` | Stock tras venta MeLi |

| `app/core.py` → `configurar_ia()` / `obtener_respuesta_ia()` | IA para endpoint `/chat` en :8080 |

| `app/utlis.py` → `refrescar_token_meli()` | Token MeLi |

| `app/utlis.py` → `enviar_whatsapp_reporte()` | Alertas WhatsApp (reintentos 503 / red) |

| `app/utlis.py` → `jid_grupo_preventa_wa` / `jid_grupo_postventa_wa` | JIDs desde `.env` (`GRUPO_PREVENTA_WA`, `GRUPO_POSTVENTA_WA`) |

| `app/utlis.py` → `meli_postventa_*` | ID y texto para alertas posventa MeLi |

| API de MeLi (externa) | Preguntas, respuestas, órdenes, ítems |

¿Por qué se hizo así?

Respuesta inmediata + procesamiento en segundo plano: MeLi tiene un timeout muy corto para los webhooks. Si el servidor demora en responder, MeLi asume que el servidor está caído y reintenta, generando respuestas duplicadas. Al separar la recepción (instantánea) del procesamiento (lento), se evita este problema.

****Servidor separado en puerto 8080:**** Permite desplegar este webhook en una URL diferente al agente principal, o correrlos en servidores distintos si se necesita escalar.

3. `app/core.py`

¿Para qué sirve?

Este es el ****cerebro del agente conversacional**** (WhatsApp, API `/chat`, menú CLI): ****Anthropic Claude**** con tool-calling, reglas de "Hugo García" e historial por usuario. La ****preventa MeLi**** con ficha técnica usa ****Gemini**** en `meli\_preventa.py`, aparte de este módulo.

****Analogía:**** Si el proyecto fuera una persona, `core.py` sería el cerebro: contiene la personalidad, los conocimientos, la memoria de la conversación y la capacidad de decidir qué hacer ante cada mensaje.

Explicación línea por línea

Importaciones de herramientas

```
from app.tools.memoria import query_sqlite, query_vector_db
from app.services.google_services import leer_datos_hoja
from app.services.siigo import *
from app.services.meli import (
    aprender_de_interacciones_meli,
    consultar_devoluciones_meli,
    ...
)
from app.tools.system_tools import (
    enviar_email_reporte,
    listar_archivos_proyecto,
    ...
)
from app.sync import (
    sincronizar_manual_por_id,
    sincronizar_inteligente,
    ...
)
```

Cada función importada aquí es una ****herramienta que el agente puede usar****. ****Claude**** no las ejecuta por arte de magia: el bucle en `obtener\_respuesta\_ia` manda cada `tool\_use` al `\_tools\_map` y reinyecta el resultado al modelo. Es la misma metáfora de la caja de herramientas, con implementación Anthropic en lugar de Gemini.

`from app.services.siigo import \*` importa ****todo**** lo que está en `siigo.py`. El asterisco (`\*`) es conveniente pero peligroso porque puede traer funciones con el mismo nombre que las de otros módulos y sobrescribirlas sin avisar.

El Prompt Maestro `INSTRUCCIONES\_MCKENNA`

```
INSTRUCCIONES_MCKENNA = """
Rol: Hugo García (McKenna Group). Operador Ejecutivo de ventas...
"""
```

Este texto largo es el ****system prompt**** o "instrucciones del sistema". Se envía en cada llamada a Claude junto con el historial. Define:

- ****Quién es el agente:**** Hugo García, vendedor ejecutivo de McKenna Group

- **Reglas antibucle:** Le prohíbe ejecutar sincronizaciones cuando no se le piden, para no desperdiciar tokens
- **Tono:** Directo, colombiano, sin rodeos
- **Reglas de ventas:** Cómo saludar, cómo consultar inventario (sin revelar stock exacto), cómo cotizar paso a paso
- **Reglas de herramientas:** Qué herramientas usar en cada situación

¿Por qué es importante? Sin este prompt, el modelo sería genérico. Con él, Claude adopta el rol de Hugo García con las reglas de negocio McKenna.

Cliente e historial (implementación actual)

- **cliente_ia:** instancia de `anthropic.Anthropic`(`ANTHROPIC_API_KEY`)`. Si falla la configuración, queda en `None`` y `obtener_respuesta_ia`` devuelve mensaje de mantenimiento.
- **_historiales:** diccionario `usuario_id → mensajes`` en memoria (se recortan a los últimos 40 intercambios). Reiniciar el proceso borra el contexto.
- **modelo_ia:** nombre legacy; el flujo activo no depende de él.

`configurar\_ia(app)`

Carga la clave Anthropic, arma `todas_las_herramientas``, genera `_tools_schema`` (JSON schema por docstring) y `_tools_map`` (`nombre_función → callable``), y concatena `INSTRUCCIONES_MCKENNA`` con casos especiales. Incluye entre otras: memoria (SQLite/Chroma), Sheets, MeLi (consultas/aprendizaje), Siigo (cotización/factura), `app/sync.py`` (facturas), `sincronizar_precios_meli_sheets``, `generar_catalogo_pdf``, `generar_guias_masivas_web``, `publicar_contenido_redes_sociales_ia``, `procesar_facturas_para_importar_productos``, tarifas envío, `buscar_producto_completo``, y herramientas de `system_tools``. La skill de stock hacia la web (`sincronizar_productos_pagina_web``) vive en `app/tools/`` y la usa `sync.py/CLI``; no está en la lista de Claude en la versión actual del código (ver `app/core.py``).

`obtener\_respuesta\_ia(pregunta, usuario\_id, historial=None)`

1. Arma la lista `messages`` (historial previo + usuario con prefijo `Usuario_{id}:``).
2. Bucle `messages.create`` con `claude-sonnet-4-6`` y `stop_reason``:
 - **tool_use:** ejecuta cada herramienta, recorta salida a 8192 caracteres, antepone `[TOOL_ERROR]`` si falla, y continúa el bucle.
 - **end_turn:** texto final + persistencia en `_historiales``.
 - **max_tokens:** guarda también el turno parcial en `_historiales``.
3. Reintentos ante sobrecarga / 503; `BadRequestError`` limpia historial del usuario.

El endpoint HTTP **POST /chat`** (8081 y duplicado en 8080) exige en el JSON el campo **session\_id`** o **usuario_id`** para no mezclar hilos entre clientes.

Conexiones con otros archivos

| Este archivo es usado por... | ¿Para qué? |

|---|---|

| `agente_pro.py`` | Inicializar la IA al arrancar |

| `app/routes.py`` | Procesar mensajes de WhatsApp |

| `app/cli.py`` | Procesar mensajes del menú de consola |

| `webhook\_meli.py` | `configurar\_ia()` + `POST /chat` en el proceso :8080 |

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `app/services/\*`, `app/tools/memoria.py`, `app/tools/system\_tools.py`, `app/tools/sincronizar\_precios.py`, scripts expuestos como tools | Herramientas registradas explícitamente en `todas\_las\_herramientas` |

| `app/sync.py` | Sync de facturas MeLi ↔ Siigo |

¿Por qué se hizo así?

****Tool loop explícito:**** Anthropic devuelve `tool\_use` o texto; el código ejecuta herramientas y vuelve a llamar al modelo hasta `end\_turn` o límites de seguridad (`MAX\_TOOL\_ITERS`). El trade-off es complejidad en `core.py`, pero control fino de errores y costo.

****Historial en RAM:**** Si el proceso reinicia, se pierde el contexto salvo que otro sistema lo persista. Por eso `/chat` requiere un `session\_id` estable por cliente.

****Todas las herramientas en un solo lugar:**** Centralizar en `core.py` facilita registrar o quitar capacidades: importar la función y añadirla a `todas\_las\_herramientas`.

4. `app/routes.py`

¿Para qué sirve?

Define las ****puertas de entrada**** del servidor web. Incluye:

- `/whatsapp` — webhook principal (****bot-mckenna**** en :3000 reenvía aquí en :8081)
- `/status`, `/panel`, `/chat` (Bearer), `/sync/\*`, `/consultar/producto`, etc.

Comandos según ****JID**** del grupo: contabilidad (`ok`/`no` + 3 dígitos, modo humano, etc.); ****preventa**** (`GRUPO\_PREVENTA\_WA`): `resp ...` / `resp preventa ...`; ****postventa**** (`GRUPO\_POSTVENTA\_WA`): `posventa <código>: <texto>` para responder en el hilo MeLi (estado en `app/data/mensajes\_posventa\_pendientes.json`); aprobación de borrador IA posventa con `hugo dale ok <order\_id>` (alerta típica al grupo postventa). Grupo ****pedidos web**** (`GRUPO\_PEDIDOS\_WEB\_WA`) para facturar/despacho vía `app/tools/web\_pedidos.py`. Referencia de JIDs: `app/data/grupos\_whatsapp\_oficiales.json`.

****Analogía:**** Es la recepción del edificio. Cada ventanilla atiende un tipo diferente de visitante y los dirige al lugar correcto.

Explicación línea por línea

```
borradores_aprobacion = {}
```

Un diccionario en memoria RAM que actúa como sala de espera temporal. Cuando la IA genera una respuesta de posventa o se recibe un comprobante de pago, se guarda aquí hasta que un humano la aprueba. La clave es el ID del cliente o de la orden, y el valor es el mensaje en espera.

****Limitación:**** Si el servidor se reinicia, este diccionario se borra y se pierden todos los borradores pendientes. El código tiene un comentario `TODO` indicando que debería usarse Redis o una base de datos.

```
#### Función `register_routes(app)`  
def register_routes(app):
```

Recibe la instancia de Flask y le añade las rutas. Este patrón (en lugar de usar `@app.route` directamente) permite que `routes.py` no tenga que importar `app` como global, evitando importaciones circulares.

```
#### Endpoint `POST /whatsapp`
```

Esta función tiene cuatro flujos posibles dependiendo del tipo de mensaje recibido:

****Flujo 1 — Aprobación de pago:****

```
if message_text.lower().startswith("pago ok"):  
    target_sender = message_text.split()[-1]  
    if target_sender in borradores_aprobacion:  
        borradores_aprobacion.pop(target_sender)  
        return jsonify({"status": "success", "respuesta": "¡Perfecto! ..."})
```

Si el mensaje dice `"pago ok 573001234567"`, el operador humano está confirmando que el pago de ese número fue válido. Se elimina de `borradores\_aprobacion` (el cliente ya no está en espera) y se responde con confirmación. `message\_text.split()[-1]` separa el mensaje por espacios y toma el último elemento (el número de teléfono).

****Flujo 2 — Detección de comprobante de pago:****

```
if has_media and media_type == 'image':  
    keywords_pago = ["bancolombia", "tarjeta", "nequi", ...]  
    is_payment = any(keyword in message_text.lower() for keyword in keywords_pago)  
    if is_payment or message_text == "":  
        borradores_aprobacion[sender_id] = "esperando_validacion_pago"  
        mensaje_aprobacion = (f"■ *COMPROBANTE DE PAGO RECIBIDO*\n ...")  
        enviar_whatsapp_reporte(mensaje_aprobacion)
```

Si llega un mensaje con imagen y menciona palabras relacionadas con pagos (o si la imagen viene sin texto), se asume que es un comprobante. Se pone al cliente en "sala de espera", se notifica al equipo por WhatsApp para que valide manualmente, y se le dice al cliente que están revisando.

****Flujo 3 — Aprobación de respuesta de posventa:****

```
if message_text.lower().startswith("hugo dale ok"):  
    target_order_id = message_text.split()[-1]  
    if target_order_id in borradores_aprobacion:  
        message_to_send = borradores_aprobacion.pop(target_order_id)  
        resultado_envio = responder_mensaje_posventa(target_order_id, message_to_send)
```

Cuando el operador escribe `"hugo dale ok 1234567890"`, aprueba el envío de la respuesta de posventa que la IA generó para esa orden. Se saca el borrador de `borradores\_aprobacion` y se envía a MeLi.

****Flujo 4 — Mensaje normal:****

```
respuesta_ia, _ = obtener_respuesta_ia(message_text, sender_id)  
  
if is_after_sale:  
    borradores_aprobacion[order_id] = respuesta_ia  
    enviar_whatsapp_reporte(mensaje_aprobacion)  
    return jsonify({"status": "waiting_for_approval", ...})  
else:  
    return jsonify({"status": "success", "respuesta": respuesta_ia})
```

Para mensajes normales, se obtiene la respuesta de la IA. Si es un mensaje de posventa (`is\_after\_sale=True`), no se envía automáticamente: se guarda como borrador y se notifica al equipo para revisión humana. Si es un mensaje de ventas normal, se responde de inmediato.

Endpoint `GET /status`

```
@app.route('/status', methods=['GET'])
def status():
    return jsonify({
        "estado": "activo",
        "timestamp": datetime.now().isoformat(),
        "servicios": {
            "mercadolibre": os.path.exists("credenciales_meli.json"),
            "google": os.path.exists("credenciales_google.json"),
            "siigo": os.path.exists("credenciales_SIIG0.json")
        },
        "version": "1.0.0"
    })
```

Endpoint de salud. Verifica si los archivos de credenciales existen (no si las credenciales son válidas, solo si el archivo está presente). Útil para monitoreo externo: si `curl http://localhost:8081/status` devuelve `200`, el servidor está vivo.

Endpoint `POST /chat`

```
token = request.headers.get('Authorization', '').replace('Bearer ', '')
if token != os.getenv('CHAT_API_TOKEN', ''):
    return jsonify({"error": "No autorizado"}), 401
```

Autenticación básica con token Bearer. El token debe pasarse en el header HTTP `Authorization: Bearer <token>`. Si no coincide con la variable `CHAT\_API\_TOKEN` del `.env`, rechaza la petición con error 401.

Conexiones con otros archivos

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `app/core.py` → `obtener\_respuesta\_ia()` | Generar respuesta de la IA |

| `modulo\_posventa.py` → `responder\_mensaje\_posventa()` | Enviar respuesta aprobada a MeLi |

| `app/utils.py` → `enviar\_whatsapp\_reporte()` | Notificar al equipo |

¿Por qué se hizo así?

****Flujo de aprobación humana:**** Para mensajes de posventa, la IA no responde automáticamente. Esto es una decisión de negocio importante: una respuesta incorrecta a un cliente que acaba de comprar puede dañar la reputación. El humano tiene la última palabra.

****Palabras clave como comandos:**** Los comandos `"pago ok"`, `"pago no"`, `"hugo dale ok"` son comandos que el operador envía desde WhatsApp al mismo número del bot. Es una forma ingeniosa de crear un flujo de aprobación sin necesidad de construir una interfaz web separada.

5. `app/cli.py`

¿Para qué sirve?

Es la **interfaz de consola** (menú de texto) que el operador usa desde la terminal. Permite ejecutar operaciones de negocio escribiendo un número, sin necesidad de abrir un navegador o una app. Corre en paralelo con el servidor web gracias al hilo secundario iniciado en `agente_pro.py`.

****Analogía:**** Es el panel de control en la sala del servidor. El servidor web atiende clientes externos, y este menú es para el operador interno que necesita hacer mantenimiento o consultas directas.

...

Menú principal (8 opciones)

■ ■ ■ CENTRO DE MANDO – McKenna Group S.A.S.

1. ■ CHAT Conversa directo con el Agente (Hugo IA)
2. ■ FACTURAS Sync facturas MeLi ↔ Siigo – elige período
3. ■ STOCK Reporte y sync de inventario entre plataformas
4. ■ CONSULTA Busca un producto en el catálogo de Google Sheets
5. ■ APRENDIZAJE Fuerza aprendizaje de Q&A recientes en MeLi
6. ■ COMPRAS Registra facturas de compra en SIIIGO desde Gmail
7. ■ CIENCIA Genera contenido científico y publica en WordPress
8. ■ SALIR Apagar el Centro de Mando

Las opciones con múltiples variantes (2 y 3) tienen ****submenús**** propios que se despliegan al seleccionárlas. Siempre se puede volver al menú principal con `[0]`.

...

Explicación línea por línea

```
import time
from app.sync import (
    sincronizar_inteligente,
    sincronizar_facturas_recientes,
    ejecutar_sincronizacion_y_reporte_stock,
    sincronizar_manual_por_id,
    sincronizar_por_dia_especifico,
)
from app.services.google_services import leer_datos_hoja
from app.services.meli import aprender_de_interacciones_meli
from app.tools.verificacion_sync_skus import verificar_sync_skus
from app.tools.sincronizar_productos_pagina_web import sincronizar_productos_pagina_web
from app.core import obtener_respuesta_ia
```

Importa las funciones de negocio que el operador ejecuta desde el menú. ****WooCommerce fue retirado****; el stock de tienda se alinea con la ****página web**** vía API (`'WEB_API_URL' / 'WEB_API_KEY'`) y ``sincronizar_productos_pagina_web``.

— — —

Función `mostrar\_menu()`

```
def mostrar_menu():
    W = 62
    print("\n" + "■"*W)
    print("  ■■  CENTRO DE MANDO – McKenna Group S.A.S.")
```

```

print("■"*W)
print(" 1. ■ CHAT          Conversa directo con el Agente (Hugo IA)")
...

```

Solo imprime texto formateado en pantalla. No tiene lógica. `W = 62` define el ancho de las líneas decorativas en un solo lugar — si se quiere cambiar el ancho, basta con cambiar ese número.

Función `\_submenu\_facturas()`

Agrupar todas las variantes de sincronización de facturas MeLi ↔ Siigo en un solo lugar. Al seleccionar la opción 2 desde el menú principal, se despliega este submenú:

```

[1] Inteligente      Cruza pendientes MeLi vs Siigo (recomendado)
[2] Últimas 24 h     Facturas emitidas en el último día
[3] Últimos N días   Tú ingresas cuántos días atrás buscar
[4] Fecha exacta     Tú ingresas la fecha (AAAA-MM-DD)
[5] Por Pack ID      Sincroniza una venta/pack específico
[0] Volver al menú principal

```

- **Inteligente** → `\_sincronizar\_inteligente()` : cruza las órdenes MeLi sin factura contra Siigo y sube los PDFs.
- **Últimas 24h** → `\_sincronizar\_facturas\_recientes(dias=1)` : caso más común para el cierre diario.
- **Últimos N días** → `\_sincronizar\_facturas\_recientes(dias=N)` : el operador ingresa cuántos días.
- **Fecha exacta** → `\_sincronizar\_por\_dia\_especifico("AAAA-MM-DD")` : útil para re-procesar un día específico.
- **Por Pack ID** → `\_sincronizar\_manual\_por\_id(pack\_id)` : para casos puntuales de una sola venta.

Función `\_submenu\_stock()`

Agrupar las operaciones de inventario. Al seleccionar la opción 3:

```

[1] Reporte completo   Sync total MeLi y reporte de stock por WhatsApp
[2] Verificar SKUs     Auditoría SKUs MeLi / SIIGO (y web cuando aplica)
[3] Sincronizar Web     Catálogo/stock MeLi → API página web (ver `_sincronizar_productos_pagina_web`)
[0] Volver al menú principal

```

- **Reporte completo** → `\_ejecutar\_sincronizacion\_y\_reporte\_stock()`.
- **Verificar SKUs** → `\_verificar\_sync\_skus(notificar\_wa=True)`.
- **Sincronizar Web** → `\_sincronizar\_productos\_pagina\_web(...)` (en el código actual la opción 3 puede usar datos de prueba hasta conectar el listado real desde MeLi).

Función `\_iniciar\_cli()`

```

def iniciar_cli():
    time.sleep(2)

```

Pausa 2 segundos antes de mostrar el menú. Esto asegura que Flask haya terminado de iniciar y que el mensaje `■ Cerebro del Agente (IA) configurado y listo.` aparezca antes que el menú, para que la consola se vea ordenada.

```

import sys
if not sys.stdin.isatty():
    print("[CLI] Sin terminal interactiva (systemd), menú deshabilitado.")
    return

```

Cuando el proceso corre como servicio de systemd (sin terminal real), `sys.stdin.isatty()` retorna `False`. En ese caso, el menú se desactiva automáticamente porque no hay nadie escribiendo en el teclado — el `input()` fallaría.

```

while True:
    mostrar_menu()
    opcion = input(" Seleccione una opción (1-8): ").strip()

```

Bucle infinito que muestra el menú y espera que el operador escriba un número. El `input()` bloquea el hilo hasta que se presiona Enter. `.strip()` elimina espacios o saltos de línea accidentales antes de comparar.

****Opción 1 — Modo chat:****

```

if opcion == "1":
    sesion_historial = []
    while True:
        user_input = input("■ Tú: ")
        if user_input.lower() in ["salir", "exit", "menu", "volver"]:
            break
        respuesta, nuevo_historial = obtener_respuesta_ia(
            pregunta=user_input,
            usuario_id="usuario_terminal_cli",
            historial=sesion_historial
        )
        if nuevo_historial:
            sesion_historial = nuevo_historial
        print(f"\n■ Agente: {respuesta}\n")

```

Entra en un sub-bucle de chat. `sesion\_historial` se reinicia cada vez que se entra al modo chat (no es persistente entre sesiones). El `usuario\_id` fijo `usuario\_terminal\_cli` le dice a la IA que el origen es la terminal, no un cliente de WhatsApp.

****Opciones 2 y 3 — Submenús:****

```

elif opcion == "2":
    _submenu_facturas()
elif opcion == "3":
    _submenu_stock()

```

Delegan a las funciones de submenú. Cuando esas funciones retornan, el bucle principal muestra el menú principal de nuevo.

****Opción 8 — Salir:****

```

elif opcion == "8":
    print("■ Apagando el Centro de Mando...")
    break

```

`break` sale del `while True` y termina la función `iniciar\_cli()`. Como el hilo CLI tenía `daemon=True`, su terminación no afecta al servidor web.

Conexiones con otros archivos

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `app/core.py` → `obtener\_respuesta\_ia()` | Modo chat (opción 1) |

| `app/sync.py` → varias funciones | Submenú de facturas (opción 2) |

| `app/tools/sincronizar\_productos\_pagina\_web.py` | Submenú stock opción 3 |

| `app/tools/verificacion\_sync\_skus.py` → `verificar\_sync\_skus()` | Submenú stock opción 2 |

| `app/services/google\_services.py` → `leer\_datos\_hoja()` | Consultar producto (opción 4) |

| `app/services/meli.py` → `aprender\_de\_interacciones\_meli()` | Aprendizaje (opción 5) |

| `\_ejecutar\_opcion\_10()` (interno) | Facturas de compra SIIGO (opción 6) |

| `\_ejecutar\_opcion\_14()` (interno) | Contenido científico WP (opción 7) |

¿Por qué se hizo así?

****Menú numérico con submenús:**** Agrupa funciones relacionadas (todas las variantes de sync de facturas juntas, todas las de stock juntas) en lugar de listarlas todas al mismo nivel. Reduce el menú principal de 14 a 8 opciones sin perder funcionalidad.

****Submenús como funciones separadas (`\_submenu\_facturas`, `\_submenu\_stock`):**** Mantiene `iniciar\_cli()` limpio y fácil de leer. Cada submenú puede crecer o modificarse sin tocar el bucle principal.

****Hilo separado:**** Si el CLI corriera en el hilo principal, el servidor web se bloquearía mientras el operador espera en el menú. Al correr en un hilo separado, ambos coexisten sin interferirse.

6. `app/services/meli.py`

¿Para qué sirve?

Contiene todas las funciones que se comunican con la ****API de Mercado Libre****. Es el "traductor" entre el proyecto y MeLi: toma pedidos en lenguaje Python y los convierte en llamadas HTTP que MeLi entiende.

****Analogía:**** Es como un intérprete en una reunión de negocios. El resto del sistema habla "Python" y MeLi habla "HTTP/REST". Este archivo es quien traduce entre ambos.

Explicación línea por línea

Función `consultar\_devoluciones\_meli()`

```
def consultar_devoluciones_meli():
    token = refrescar_token_meli()
    if not token:
        return "■ Error: No se pudo obtener el token de Mercado Libre."

    fecha_inicio = "2026-01-01T00:00:00.000-00:00"
    url = f"https://api.mercadolibre.com/orders/search?seller=me&order.date_created.from={fecha_inicio}"
    headers = {"Authorization": f"Bearer {token}"}
```

Primero renueva el token. Si falla, retorna error inmediatamente (patrón ****early return****). La `fecha\_inicio` está hardcodeda — esto significa que si el año cambia, esta fecha quedaría desactualizada y habría que actualizarla manualmente.

```
data = res.json().get('results', [])
devoluciones = [o for o in data if o.get('status') in ['cancelled', 'invalid']]
```

Filtra las órdenes usando ****list comprehension**** (una forma compacta de crear listas con condiciones). Solo guarda las órdenes cuyo estado es `cancelled` o `invalid`.

Función `consultar\_detalle\_venta\_meli(pack\_id)`

```
def consultar_detalle_venta_meli(pack_id: str):
    url = f"https://api.mercadolibre.com/orders/{pack_id}"
```

```

res = requests.get(url, headers=headers, timeout=10)
if res.status_code == 200:
    data = res.json()
    return (f"■ Venta {pack_id} encontrada.\n"
            f"- Fecha: {data.get('date_created')}\n"
            f"- Estado: {data.get('status')}\n"
            f"- Valor: ${data.get('total_amount')}")

```

Consulta los detalles de una venta específica por su ID. `timeout=10` significa que si MeLi no responde en 10 segundos, lanza un error en lugar de esperar indefinidamente. Retorna un texto formateado listo para mostrar al usuario.

Función `subir\_factura\_meli(pack\_id, pdf\_base64)`

```

def subir_factura_meli(pack_id, pdf_base64):
    pdf_puro = str(pdf_base64).strip().replace("\n", "").replace("\r", "")
    if "," in pdf_puro:
        pdf_puro = pdf_puro.split(",")[1]
    pdf_decodificado = base64.b64decode(pdf_puro)

```

El PDF llega como texto en formato base64 (base64 convierte archivos binarios en texto para poder transmitirlos como JSON). Aquí se limpia ese texto (eliminando saltos de línea y posibles prefijos como `data:application/pdf;base64,`) y luego se decodifica de vuelta a bytes binarios.

```

url = f"https://api.mercadolibre.com/packs/{pack_id}/fiscal_documents"
files = {'file': (f"Fac_{pack_id}.pdf", pdf_decodificado, 'application/pdf')}
res = requests.post(url, headers=headers, files=files, timeout=30)

```

Sube el PDF a MeLi como un archivo adjunto. El diccionario `files` le dice a `requests` que esto es una subida de archivo multipart (como cuando subes una foto en un formulario web). `timeout=30` porque las subidas de archivos pueden tardar más que una consulta simple.

Función `aprender\_de\_interacciones\_meli()`

Esta es la función más compleja del archivo. Implementa el ciclo de **aprendizaje automático** del agente.

```

chroma_client = chromadb.PersistentClient(path="./memoria_vectorial")
coleccion_experiencia = chroma_client.get_or_create_collection(name="mckenna_brain")

```

Conecta a ChromaDB, una base de datos vectorial que almacena texto de forma que se puede buscar por similitud semántica (no solo por palabras exactas). La colección "mckenna_brain" es donde vive el conocimiento aprendido.

```

url = "https://api.mercadolibre.com/my/received_questions/search?status=ANSWERED&limit=15"
preguntas = res.json().get('questions', [])
texto_bruto = "Historial de interacciones recientes con clientes en Mercado Libre:\n"
for q in preguntas:
    texto_bruto += f"- Pregunta del cliente: {q.get('text')}\n - Nuestra respuesta: {q.get('answer', {})}\n"

```

Descarga las últimas 15 preguntas respondidas de MeLi y las convierte en un texto continuo que incluye tanto la pregunta como la respuesta.

```

prompt = (
    f"Actúa como un técnico en farmacología experto... "
    f"Identifica los patrones, las dudas más comunes..."
    f"Resume esto en un párrafo conciso como una 'lección aprendida'..."
    f"\n--- HISTORIAL ---\n{texto_bruto}"
)
response = client.models.generate_content(model='gemini-2.0-flash-lite', contents=prompt)
aprendizaje_generado = response.text

```

Envía las 15 preguntas+respuestas a Gemini Flash (un modelo más económico y rápido que el principal) y le pide que lo convierta en una sola "lección aprendida" densa.

```
doc_id = f"exp_meli_{int(time.time())}"
coleccion_experiencia.add(
    documents=[aprendizaje_generado],
    metadatas=[{"fuente": "meli_qa_auto", "fecha": str(datetime.now().date())}],
    ids=[doc_id]
)
```

Guarda la lección generada en ChromaDB con un ID único basado en el timestamp. En el futuro, cuando llegue una pregunta similar, la función `query\_vector\_db` puede recuperar esta lección para que Hugo García responda con más contexto.

Función `responder\_solicitud\_rut(order\_id)`

```
def responder_solicitud_rut(order_id):
    clean_id = str(order_id).replace("Venta #", "").strip()
    print(f"■ [MELI-RUT] Enviando mensaje de RUT a la orden: {clean_id}")
    return f"■ Solicitud de RUT procesada para la orden {clean_id}."
```

Esta función está ****incompleta**** — tiene un `TODO` en el código que indica que la lógica real de enviar el mensaje a MeLi todavía no está implementada. Por ahora solo limpia el ID y retorna un mensaje de éxito sin hacer nada real.

Función `buscar\_ventas\_acordar\_entrega(dias=3)`

```
res_me = requests.get("https://api.mercadolibre.com/users/me", headers=headers)
res_me.raise_for_status()
seller_id = res_me.json().get('id')
```

Primero obtiene el ID del vendedor consultando el endpoint `/users/me` de MeLi. `raise\_for\_status()` lanza una excepción automáticamente si el status HTTP es 4xx o 5xx, evitando tener que verificar manualmente.

```
for orden in res.get('results', []):
    shipping_info = orden.get('shipping', {}) or {}
    shipping_type = shipping_info.get('substatus') or shipping_info.get('shipping_mode')
    if orden.get('status') == 'paid' and shipping_type in ['to_agree', 'custom', 'not_specified']:
        ordenes_encontradas.append(str(orden.get('id')))
```

Filtra órdenes que están pagadas pero cuyo envío es "a acordar con el comprador" (no usan el sistema de envíos de MeLi). Estas necesitan coordinación manual con el comprador.

Conexiones con otros archivos

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `app/utils.py` → `refrescar\_token\_meli()` | Obtener token válido antes de cada llamada |

| API de MeLi (externa) | Todas las operaciones |

| ChromaDB (base de datos local) | Guardar aprendizajes |

| Gemini API (Google) | Generar resúmenes de aprendizaje |

| Es llamado desde... | ¿Para qué? |

|---|---|

| `app/core.py` | Registrar funciones como herramientas de la IA |

| `app/cli.py` | Opción 5 del menú (aprendizaje) |

| `app/sync.py` | Subir facturas a órdenes de MeLi |

¿Por qué se hizo así?

****Refrescar token antes de cada llamada:**** Los tokens de OAuth expiran. En lugar de verificar si expiró y luego refrescar, se refresca siempre antes de usarlo. Esto garantiza que nunca se haga una llamada con un token vencido, a costa de una pequeña llamada extra ocasional.

****Retornar strings en lugar de lanzar excepciones:**** Las funciones retornan mensajes de error como `"■ Error: ..."` en lugar de lanzar excepciones. Esto facilita el uso desde la IA: el modelo puede leer el mensaje de error y decidir qué hacer, mientras que una excepción no capturada rompería la conversación.

7. `app/services/siigo.py`

¿Para qué sirve?

Contiene todas las funciones que se comunican con ****SIIGO****, el sistema ERP (Enterprise Resource Planning) usado para crear facturas electrónicas oficiales, cotizaciones y gestionar documentos contables. Es el módulo más grande y crítico del proyecto porque maneja dinero real y documentos con validez ante la DIAN (entidad fiscal colombiana).

****Analogía:**** Si MeLi es la vitrina donde los clientes compran, SIIGO es el contador de la empresa que genera los recibos oficiales y los reporta al gobierno.

Explicación línea por línea

Función `autenticar\_siigo(forzar=False)`

```
def autenticar_siigo(forzar=False):
    ruta_json = os.path.expanduser("~/mi-agente/credenciales_SIIGO.json")
    with open(ruta_json, "r") as f:
        creds = json.load(f)
```

Lee las credenciales desde el archivo JSON. `os.path.expanduser("~/")` convierte `~` en la ruta completa del directorio home del usuario (ej: `/home/mckg`).

```
    if not forzar and time.time() < creds.get("token_vencimiento", 0):
        return creds["access_token"]
```

Si el token guardado todavía no ha expirado (y no se forzó el refresco), lo devuelve directamente sin hacer una llamada a SIIGO. Este es un ****caché de token****: evita autenticarse en cada llamada, lo cual sería lento e innecesario.

```
    res = requests.post(
        "https://api.siigo.com/auth",
        json={"username": creds["username"], "access_key": creds["api_key"]},
        headers={"Partner-Id": PARTNER_ID},
        timeout=10
    )
```

Si el token expiró, hace una nueva autenticación. `Partner-Id` es un header requerido por SIIGO para identificar que la llamada viene de una integración externa (no del portal web de SIIGO).

```
if res.status_code == 200:
    token = res.json().get("access_token")
    creds.update({"access_token": token, "token_vencimiento": time.time() + (23 * 3600)})
    with open(ruta_json, "w") as f:
        json.dump(creds, f)
    return token
```

Guarda el nuevo token en el mismo archivo JSON junto con su tiempo de expiración (23 horas desde ahora). Así el próximo uso no requiere autenticación.

Función `obtener\_facturas\_siigo\_paginadas(fecha\_inicio)`

```
page = 1
while True:
    res = requests.get(
        f"https://api.siigo.com/v1/invoices?created_start={fecha_inicio}&page={page}",
        ...
    )
    facturas_pagina = data.get("results")
    if facturas_pagina:
        todas_las_facturas.extend(facturas_pagina)
        if not data.get("pagination") or data["pagination"]["total_results"] == len(todas_las_facturas):
            break
        page += 1
    else:
        break
```

La API de SIIGO devuelve las facturas de a páginas (por ejemplo, 50 por página). Este bucle sigue pidiendo páginas hasta que el total de facturas descargadas sea igual al `total\_results` que SIIGO reporta. `extend()` agrega todos los elementos de una lista a otra (a diferencia de `append()` que agrega la lista como un solo elemento).

Función `descargar\_factura\_pdf\_siigo(id\_factura)`

```
res = requests.get(
    f"https://api.siigo.com/v1/invoices/{id_factura}/pdf",
    ...
)
if res.status_code == 200:
    return res.json().get("base64", "")
```

Descarga el PDF de una factura en formato base64 (texto que representa el PDF binario). Este texto luego se convierte de nuevo a binario para guardarlo como archivo o subirlo a MeLi.

Función `crear\_cotizacion\_siigo(nombre\_cliente, ...)`

```
cliente_data = {
    "person_type": "Person",
    "id_type": "13", # Cédula de ciudadanía
    "identification": identificacion,
    ...
}
```

Construye el objeto "cliente" con la estructura que espera la API de SIIGO. `id\_type: "13"` es el código de SIIGO para cédula de ciudadanía colombiana.

```
payload = {
    "document": {"id": 5804}, # Factura de Venta estándar
    "seller": 704, # Vendedor: Víctor Hugo García Barrero
    "observations": "COTIZACIÓN: Este documento es una factura en estado de borrador...",
```

```

    ...
}

```

Los valores `5804` y `704` son IDs específicos configurados en SIIGO para esta empresa. Si se cambia de empresa o de SIIGO, estos IDs cambiarían. Las observaciones dejan claro que es una cotización, no una factura real.

```

resultado_email = enviar_email_reporte("Cotización McKenna Group", mensaje_email, email)
from app.utils import enviar_whatsapp_reporte
enviar_whatsapp_reporte(mensaje_wa)

```

Después de crear la cotización en SIIGO, envía una copia por correo al cliente y notifica al equipo por WhatsApp. Nótese la importación local (`from app.utils import ...` dentro de la función) para evitar importaciones circulares entre módulos.

```

#### Función `crear_cotizacion_preliminar(nombre_cliente, ...)`

cotizacion = {
    "id_preliminar": f"PRE-{int(time.time())}",
    ...
}
os.makedirs("cotizaciones_preliminares", exist_ok=True)
file_path = f"cotizaciones_preliminares/{cotizacion['id_preliminar']}.json"
with open(file_path, "w") as f:
    json.dump(cotizacion, f, indent=4)

```

A diferencia de `crear\_cotizacion\_siigo`, esta función **no llama a la API de SIIGO**. Guarda la cotización como un archivo JSON local. El `id\_preliminar` usa el timestamp Unix (`int(time.time())`) para garantizar que sea único. `exist\_ok=True` evita un error si la carpeta ya existe.

¿Por qué existe esta función? Crear una factura en SIIGO es un proceso irreversible desde el punto de vista fiscal. Se usa esta función como "borrador" mientras el cliente confirma y paga, y solo se crea la factura real en SIIGO cuando el pago está confirmado.

```

#### Función `crear_factura_completa_siigo(cotizacion_data, comprobante_pago_path)`

```

Esta es la función más larga y la más crítica. Hace cuatro cosas encadenadas:

****Paso 1 — Crear factura en SIIGO:****

```

payload = {
    "document": {"id": 26670}, # ID de Documento para Factura Electrónica de Venta
    "customer": {
        "id_type": "13" if len(identificacion) <= 10 else "31", # 13: Cédula, 31: NIT
        "person_type": "Person" if len(identificacion) <= 10 else "Company",
        ...
    },
    ...
}

```

Detecta si el cliente es persona natural (cédula, máximo 10 dígitos) o empresa (NIT, más de 10 dígitos) para asignar el tipo correcto en SIIGO. Esto es un requisito de la DIAN colombiana.

****Paso 2 — Verificar estado ante la DIAN:****

```

time.sleep(2) # Dar tiempo a que se procese en la DIAN
res_get = requests.get(f"https://api.siigo.com/v1/invoices/{factura_id}", ...)
estado_factura = factura_siigo.get("stamp", {}).get("status", "Desconocido")
if estado_factura == "Rejected":
    observaciones_adicionales = f"\n■■■ *Estado DIAN:* RECHAZADA. Inconsistencias: {inconsistencias}"

```

Espera 2 segundos y luego consulta el estado de la factura ante la DIAN. La DIAN puede aceptar o rechazar facturas electrónicas. Si es rechazada, se incluye el motivo en el reporte.

****Paso 3 — Adjuntar comprobante de pago:****

```
if comprobante_pago_path and os.path.exists(comprobante_pago_path):
    with open(comprobante_pago_path, "rb") as f:
        encoded_file = base64.b64encode(f.read()).decode('utf-8')
    att_res = requests.post(
        f"https://api.siigo.com/v1/invoices/{factura_id}/attachments",
        json={"file_name": ..., "base64": encoded_file},
        ...
    )
```

Si se proporcionó una imagen de comprobante de pago, la convierte a base64 y la sube como adjunto a la factura en SIIGO.

****Paso 4 — Enviar reportes:****

```
enviar_whatsapp_reporte(mensaje_wa) # Resumen de texto
if pdf_path:
    enviar_whatsapp_archivo(pdf_path, ...) # PDF de la factura
if comprobante_pago_path:
    enviar_whatsapp_archivo(comprobante_pago_path, ...) # Comprobante
```

Envía tres cosas al grupo de WhatsApp del equipo: el resumen de texto, el PDF de la factura y el comprobante del cliente.

Conexiones con otros archivos

| Este archivo llama a... | ¿Para qué? |

|---|---|

| `app/tools/system\_tools.py` → `enviar\_email\_reporte()` | Enviar cotización por email |

| `app/utils.py` → `enviar\_whatsapp\_reporte()` | Notificar al equipo |

| API de SIIGO (externa) | Todas las operaciones contables |

| DIAN (implícito via SIIGO) | Validación fiscal de facturas |

| Es llamado desde... | ¿Para qué? |

|---|---|

| `app/core.py` | Registrar funciones como herramientas de la IA |

| `app/sync.py` | Obtener facturas para sincronizar con MeLi |

¿Por qué se hizo así?

****Caché de token en archivo JSON:**** Los tokens de SIIGO duran 24 horas. Guardar el token en el mismo archivo de credenciales evita tener que autenticarse en cada llamada (que sería lento). El archivo JSON actúa como un almacén de estado persistente entre ejecuciones del programa.

****Dos tipos de cotización:**** `crear\_cotizacion\_preliminar` (local) y `crear\_cotizacion\_siigo` (en el ERP) responden a diferentes momentos del proceso de venta. La local es para "apartarle al cliente" sin comprometer nada fiscalmente.

8. `app/tools/system\_tools.py`

¿Para qué sirve?

Contiene herramientas que le dan a la IA capacidades de **administración del sistema**: leer y editar código del propio proyecto, hacer backups, ejecutar scripts, y enviar correos. Son herramientas "meta" — la IA puede usarlas para modificarse a sí misma o para comunicarse.

Analogía: Si las otras herramientas son el trabajo del día a día, estas son las herramientas del taller de mantenimiento. Con ellas, el agente puede hacer ajustes al sistema mientras está corriendo.

Nota de seguridad: ``parchear_funcion`` y ``ejecutar_script_python`` son herramientas poderosas. Permiten que la IA modifique y ejecute código arbitrario. En producción, esto debe usarse con mucho cuidado.

Explicación línea por línea

```
##### Función `listar_archivos_proyecto(path)`

def listar_archivos_proyecto(path: str = '.') -> str:
    archivos = os.listdir(path)
    resultado = f"Contenido de '{path}':\n"
    for i, nombre in enumerate(sorted(os.listdir(path))):
        ruta_completa = os.path.join(path, nombre)
        es_dir = "[DIR]" if os.path.isdir(ruta_completa) else "[FILE]"
        resultado += f"{i+1}. {es_dir:6} {nombre}\n"
    return resultado
```

Lista el contenido de un directorio. ``sorted()`` ordena alfabéticamente. ``os.path.isdir()`` detecta si es una carpeta o un archivo. ``{es_dir:6}`` formatea el texto con 6 caracteres de ancho para que las columnas queden alineadas. Retorna texto plano para que la IA pueda leerlo y describírselo al usuario.

```
##### Función `crear_backup(ruta_archivo)`

def crear_backup(ruta_archivo: str) -> str:
    os.makedirs("backups", exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    nombre_base = os.path.basename(ruta_archivo)
    ruta_backup = os.path.join("backups", f"{nombre_base}.{timestamp}.bak")
    shutil.copy2(ruta_archivo, ruta_backup)
    return f"■ Éxito: Backup creado en {ruta_backup}"
```

Copia un archivo a la carpeta ``/backups/`` con el timestamp en el nombre para que nunca se sobrescriba un backup anterior. ``shutil.copy2`` copia el archivo preservando los metadatos (fecha de creación, permisos). Esta función se llama automáticamente antes de ``parchear_funcion`` como medida de seguridad.

```
##### Función `leer_funcion(ruta_archivo, nombre_funcion)`

def leer_funcion(ruta_archivo: str, nombre_funcion: str) -> str:
    with open(ruta_archivo, 'r', encoding='utf-8') as f:
        source_code = f.read()
        tree = ast.parse(source_code)
        for nodo in ast.walk(tree):
            if isinstance(nodo, ast.FunctionDef) and nodo.name == nombre_funcion:
                return f"python\n{ast.unparse(nodo)}\n"
```

Usa el módulo ``ast`` (Abstract Syntax Tree) de Python para analizar el código fuente como un árbol de objetos en lugar de texto plano. ``ast.parse()`` convierte el texto del archivo en un árbol de nodos. ``ast.walk()`` recorre todos los

nodos del árbol. ``isinstance(nodo, ast.FunctionDef)`` verifica si el nodo es una definición de función. ``ast.unparse()`` convierte el nodo de vuelta a código Python legible.

****¿Por qué usar AST en lugar de buscar con texto?**** Porque buscar la función con ``"def nombre_funcion"`` en texto podría encontrar comentarios o cadenas que contengan esas palabras. AST analiza el código como Python realmente lo interpreta, sin confusiones.

```
#### Función `parchear_funcion(ruta_archivo, nombre_funcion, nuevo_codigo_funcion)`  
  
def parchear_funcion(ruta_archivo: str, nombre_funcion: str, nuevo_codigo_funcion: str) -> str:  
    ast.parse(nuevo_codigo_funcion) # Verificar sintaxis antes de tocar el archivo  
    backup_msg = crear_backup(ruta_archivo)  
    if "■" in backup_msg:  
        return f"No se pudo aplicar el parche porque falló la creación del backup: {backup_msg}"
```

Primero verifica que el nuevo código sea sintácticamente válido con ``ast.parse()``. Si el código tiene errores de Python, ``ast.parse()`` lanza un ``SyntaxError`` que se captura más abajo. Luego crea un backup. Si el backup falla, cancela toda la operación — no tiene sentido modificar el archivo si no podemos revertirlo.

```
    with open(ruta_archivo, 'r', encoding='utf-8') as f:  
        lineas = f.readlines()  
        tree = ast.parse("".join(lineas))  
        inicio, fin = -1, -1  
        for nodo in ast.walk(tree):  
            if isinstance(nodo, ast.FunctionDef) and nodo.name == nombre_funcion:  
                inicio, fin = nodo.lineno - 1, nodo.end_lineno  
                break
```

Busca la función en el árbol AST para obtener los números de línea exactos donde empieza (``lineno``) y termina (``end_lineno``). Se resta 1 a ``lineno`` porque las líneas en Python empiezan en 1 pero los índices de lista empiezan en 0.

```
    nuevas_lineas = lineas[:inicio] + [nuevo_codigo_funcion + "\n"] + lineas[fin:]  
    with open(ruta_archivo, 'w', encoding='utf-8') as f:  
        f.writelines(nuevas_lineas)
```

Reemplaza las líneas de la función antigua con el nuevo código usando slicing de listas: toma todo lo que está antes de la función, agrega el nuevo código, y agrega todo lo que está después. Luego escribe el resultado de vuelta al archivo.

```
#### Función `crear_nuevo_script(nombre_archivo, codigo_completo)`  
  
def crear_nuevo_script(nombre_archivo: str, codigo_completo: str) -> str:  
    if not nombre_archivo.endswith('.py'):  
        return "■ Error: El nombre debe terminar en '.py'."  
    ast.parse(codigo_completo)  
    with open(nombre_archivo, 'w', encoding='utf-8') as f:  
        f.write(codigo_completo)  
    return f"■ Script '{nombre_archivo}' creado."
```

Crea un nuevo archivo Python. Valida que el nombre termine en ``'.py`` y que el código sea sintácticamente válido antes de crear el archivo. Nótese que no crea un backup porque el archivo es nuevo (no hay nada que respaldar).

```
#### Función `ejecutar_script_python(nombre_archivo)`  
  
def ejecutar_script_python(nombre_archivo: str) -> str:  
    resultado = subprocess.run(  
        [sys.executable, nombre_archivo],  
        capture_output=True,  
        text=True,
```

```

        timeout=45,
        check=False
    )
    salida = f"--- Salida de {nombre_archivo} ---\n{resultado.stdout}\n"
    if resultado.stderr:
        salida += f"--- Errores de {nombre_archivo} ---\n{resultado.stderr}\n"
    salida += f"--- Código de salida: {resultado.returncode} ---"
    return salida

```

• ``sys.executable`` es la ruta al intérprete de Python actual (el mismo que está corriendo el agente). Esto asegura que el script use el mismo Python y el mismo entorno virtual.

• ``capture_output=True`` captura ``stdout`` y ``stderr`` en lugar de imprimirlos en la consola.

• ``text=True`` convierte la salida de bytes a string.

• ``timeout=45`` mata el proceso si tarda más de 45 segundos.

• ``check=False`` no lanza excepción si el script falla (el error queda en ``returncode``).

La función captura tanto la salida normal como los errores y los retorna juntos para que la IA pueda interpretarlos.

Función ``enviar_email_reporte(destinatario, asunto, cuerpo)``

```

def enviar_email_reporte(destinatario: str, asunto: str, cuerpo: str) -> str:
    remitente = os.getenv("EMAIL_SENDER")
    password = os.getenv("EMAIL_PASSWORD")

    msg = MIMEMultipart()
    msg['From'] = remitente
    msg['To'] = destinatario
    msg['Subject'] = asunto
    msg.attach(MIMEText(cuerpo, 'plain'))

    server = smtplib.SMTP('smtp.gmail.com', 587)
    server.starttls()
    server.login(remitente, password)
    server.send_message(msg)
    server.quit()

```

Envía un correo usando Gmail como servidor SMTP. ``MIME`` (Multipurpose Internet Mail Extensions) es el estándar para formatear correos. ``MIMEMultipart`` permite adjuntar múltiples partes (texto, archivos). ``starttls()`` activa el cifrado TLS para que la contraseña no viaje en texto plano. ``587`` es el puerto estándar para SMTP con TLS.

****Requisito:**** En Gmail debe activarse "Acceso de apps menos seguras" o crear una "Contraseña de aplicación" para que el login funcione.

Función ``enviar_reporte_controlado(mensaje)``

```

def enviar_reporte_controlado(mensaje):
    print("\n" + "█"*40 + f"\n█ REPORTE GENERADO:\n{mensaje}\n" + "█"*40)
    if True: # ← Bug conocido
        from app.utils import enviar_whatsapp_reporte
        return enviar_whatsapp_reporte(mensaje)
    print("Envío cancelado por el usuario.") # ← Esta línea nunca se ejecuta
    return False

```

Esta función tiene un ****bug evidente****: la condición ``if True:`` siempre es verdadera, así que el ``print("Envío cancelado")`` y el ``return False`` son código inalcanzable (dead code). El comentario ``TODO`` en el código indica que originalmente debía pedir confirmación con ``input()``, pero fue simplificado. Actualmente siempre envía el mensaje sin preguntar.

Conexiones con otros archivos

```
| Este archivo llama a... | ¿Para qué? |
|---|---|
| `app/utils.py` → `enviar_whatsapp_reporte()` | Enviar reportes por WhatsApp |
| Es llamado desde... | ¿Para qué? |
|---|---|
| `app/core.py` | Registrar funciones como herramientas de la IA |
| `app/services/siigo.py` → `crear_cotizacion_siigo()` | Enviar email de cotización |
---
```

¿Por qué se hizo así?

****AST en lugar de regex para el código:**** Manipular código fuente con expresiones regulares es frágil (¿qué pasa si hay una función con el mismo nombre dentro de otra clase? ¿O dentro de un string multiline?). AST entiende la estructura real del código Python y no se confunde con estos casos.

****Backup obligatorio antes de parchear:**** Cualquier modificación de código en producción sin red de seguridad es peligrosa. Al forzar el backup, siempre hay un punto de restauración si el parche rompe algo.

****Timeout en la ejecución de scripts:**** Sin timeout, un script con un bucle infinito bloquearía el hilo del agente para siempre. 45 segundos es suficiente para la mayoría de las tareas de negocio pero evita bloqueos indefinidos.

10. Nuevas Skills (Herramientas autónomas — Claude)

¿Para qué sirven?

Varias funciones en `app/tools/` y servicios se registran en `app/core.py` como herramientas de ****Claude**** (tool use). La ****preventa MeLi**** con ficha no usa este mecanismo: va por `preventa\_meli.py` + ****Gemini**** en `meli\_preventa.py`.

Arquitectura

`\_fn\_to\_tool\_schema()` convierte docstrings + type hints en esquema JSON. El bucle en `obtener\_respuesta\_ia` ejecuta cada `tool\_use` y devuelve el resultado al modelo.

Ejemplos (no exhaustivo)

```
| Skill (nombre función) | Archivo / módulo | Descripción |
|---|---|---|
| `sincronizar_precios_meli_sheets` | `app/tools/sincronizar_precios.py` | Precios MeLi → Google Sheets |
| `generar_catalogo_pdf` | `generar_catalogo.py` | PDF catálogo con fotos MeLi |
| `publicar_contenido_redes_sociales_ia` | `pipeline_contenido_facebook.py` | Pipeline multimedia → Facebook |
| `generar_guias_masivas_web` | `generar_guias_masivas.py` | Guías JSON para el sitio |
```

| `sincronizar\_productos\_pagina\_web` | `app/tools/sincronizar\_productos\_pagina\_web.py` | Stock/precios hacia la API de la tienda web (usada desde `sync.py`/CLI; comprobar si está enlazada en `todas\_las\_herramientas`) |

****Nota:**** Los scripts `sincronizar\_wc\_desde\_meli.py` y `woocommerce` fueron retirados; la tienda es la ****página web**** propia (`PAGINA\_WEB/site/`) con API (`WEB\_API\_URL`, `WEB\_API\_KEY`).

11. Observabilidad, auditoría de scripts, backup y Git

11.1 Trazabilidad (`app/observability.py`)

- Cada request en Flask (8081 y 8080) obtiene un ****request_id**** (cabecera opcional `X-Request-ID` o UUID).
- Los hilos en segundo plano (MeLi, WhatsApp) usan ****spawn_thread()**** para conservar el mismo id en logs.
- Con ****AGENTE_LOG_JSON=1**** se imprimen líneas JSON (eventos http, herramientas Claude, inicio de turno IA).
- ****GET /status**** devuelve también `request\_id` para depuración rápida.

11.2 Herramientas que tocan archivos (`app/tools/system\_tools.py`)

Si ****AGENTE_RESTRICT_FILE_TOOLS=1**** o ****FLASK_ENV=production****, las funciones ****parchear_funcion****, ****crear_nuevo_script**** y ****ejecutar_script_python**** solo actúan sobre rutas bajo los prefijos de ****AGENTE_FILE_TOOL_PREFIXES**** (por defecto `scripts/`, `app/tools/`, `tests/`). Así se reduce el riesgo en producción.

11.3 Auditoría de scripts

- ****app/data/scripts_manifest.json****: lista de rutas `.py` a comprobar.
- ****app/tools/script_audit.py****: `ejecutar\_auditoria\_dict()` / herramienta del agente ****auditar_scripts**** — compila con `py\_compile` sin ejecutar el programa.
- ****scripts/auditar_scripts_cron.py****: pensado para cron; si hay errores, envía WhatsApp (salvo ****AGENTE_AUDITORIA_SKIP_WA=1****).
- ****scripts/instalar_cron_mckenna.sh****: instala de forma idempotente el bloque cron (p. ej. 7:15 diario, log en `log\_cron.txt`).

11.4 Backup nocturno y GitHub (`app/tools/backup\_drive.py`)

- A las ****2:00**** (hilo daemon arrancado desde `agente\_pro`): genera `backup\_hugo\_\*.tar.gz` en ****backups_drive/**** (carpeta ****ignorada por git****), opcionalmente sube a Google Drive si hay `DRIVE\_BACKUP\_FOLDER\_ID` y service account.
- Después intenta ****git add -A****, ****commit**** y ****git push origin <rama_actual>**** si hay cambios. Desactivar con ****AGENTE_NIGHTLY_GIT_PUSH=0****.
- El mensaje de WhatsApp incluye resultado del backup y resumen del intento de Git.

11.5 Grupo WhatsApp de alertas

- ****GRUPO_ALERTAS_SISTEMAS_WA**** (y ****jid_grupo_alertas_sistemas_wa()**** en `app/utlis.py`): mismo destino para ****avisos de backup nocturno**** y ****fallos de la auditoría por cron****. Puedes anular solo el cron con ****GRUPO_AUDITORIA_SCRIPTS_WA****.

11.6 Manual PDF y correo

- Generar: ``source venv/bin/activate && python3 scripts/generar_manual.py``
- Generar y enviar a `**cynthua0418@gmail.com**`: añadir `**`--enviar`**` (requiere ``EMAIL_SENDER`` / ``EMAIL_PASSWORD`` en ``.env``).

11.7 Memoria vectorial de bugs resueltos (debugging agentico)

- Dataset histórico: ``app/data/debugging_resuelto.jsonl`` (JSONL append-only).
- Script de guardado: ``scripts/guardar_memoria_debug.py``.
- Script de consulta: ``scripts/consultar_memoria_debug.py``.
- Colección Chroma: ``mckenna_debug_memory`` (en ``memoria_vectorial/``).
- Skill interna: ``.agents/skills/debug-memory/SKILL.md``.

****Guardar un bug resuelto****

```
source venv/bin/activate
python3 scripts/guardar_memoria_debug.py \
  --titulo "OAuth MeLi no JSON" \
  --problema "Refresco token falla intermitente" \
  --causa-raiz "Respuesta upstream invalida y sin validacion" \
  --solucion "Agregar parse defensivo + alerta WA con cooldown" \
  --archivos "app/utls.py,scripts/diagnostico_preventa_postventa_wa.py" \
  --tags "meli,oauth,alertas,debug" \
  --fuente "chat-actual"
```

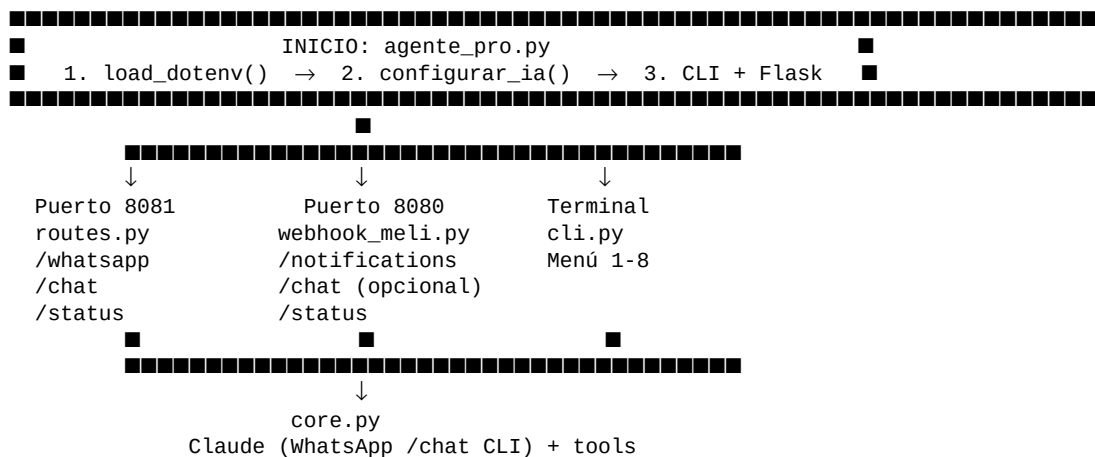
****Consultar memoria****

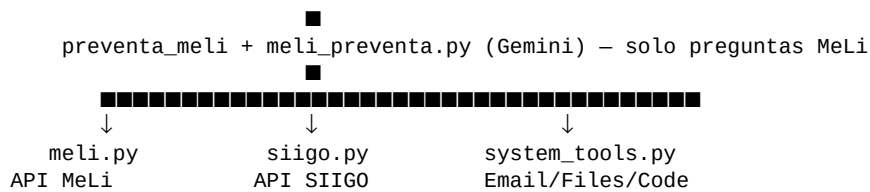
```
source venv/bin/activate
python3 scripts/consultar_memoria_debug.py --q "preventa sin ficha por matching titulo" --n 5
```

Cómo agregar una skill a Claude

1. Función con docstring y tipos en ``app/tools/`` (o reutilizar módulo existente).
2. Importar en ``app/core.py`` y añadir a ``todas_las_herramientas`` dentro de ``configurar_ia()``.

RESUMEN DE FLUJO COMPLETO





 Documento técnico del repositorio McKenna Group Agente. Última actualización: 2026-04-14 (sección 0: actualización de repositorio + ajustes de manual).

9. APIs de Generación de Contenido Multimedia (consola)

> ****Nota:**** Estas integraciones ****no forman parte del menú del Centro de Mando****. Se usan directamente desde la terminal con scripts independientes. Se documentan aquí como recordatorio de que ya están implementadas y listas para usar cuando se necesite crear contenido.

APIs integradas

| API | Proveedor | Para qué se usa |

|-----|-----|-----|

| ****Ideogram**** | Ideogram AI | Generación de imágenes con texto — fondos visuales para infografías |

| ****ElevenLabs**** | ElevenLabs | Síntesis de voz (TTS) — narración en español colombiano para videos |

| ****fal.ai / Kling**** | fal.ai | Generación de video desde imagen o texto (Kling text-to-video v1.6) |

| ****PIL / Pillow**** | Python | Composición de infografías — superpone texto y logo sobre las imágenes |

| ****ffmpeg**** | ffmpeg | Fallback de video local — efecto Ken Burns y concatenación de clips |

| ****Facebook Graph API**** | Meta | Publicación de fotos, posts y videos en la página de McKenna Group |

Scripts disponibles

| Script | Comando | ¿Qué hace? |

|-----|-----|-----|

| `pipeline\_contenido\_facebook.py` | `python3 pipeline\_contenido\_facebook.py --tipo ficha --slug acido-ascorbico` |

Pipeline completo: copy (Gemini) → imagen (Ideogram) → voz (ElevenLabs) → video (Kling) → publica en Facebook |

| `generar\_infografias\_facebook.py` | `python3 generar\_infografias\_facebook.py --tipo ficha --n 3` | Genera infografías estáticas con PIL y las publica en Facebook (sin video ni audio) |

| `sincronizar\_facebook.py` | `python3 sincronizar\_facebook.py` | Limpia la página de Facebook y la republica con productos, guías y blog posts actuales |

Tipos de contenido del pipeline

- ****ficha**** — Infografía de un ingrediente (beneficios, concentración, compatibilidad)
- ****receta**** — Fórmula paso a paso con ingredientes visuales
- ****comparativa**** — Dos ingredientes frente a frente

- `**tip**` — Consejo profesional de formulación
- `**--auto**` — El sistema elige automáticamente qué contenido publicar a continuación

Credenciales requeridas (en `.env`)

```

GOOGLE_API_KEY      # Gemini — generación de copy
IDEOGRAM_API_KEY    # Ideogram — generación de imágenes
ELEVENLABS_API_KEY   # ElevenLabs — síntesis de voz
FAL_KEY              # fal.ai — generación de video con Kling
FB_PAGE_TOKEN        # Facebook Graph API — publicación en página
FB_PAGE_ID           # ID de la página de Facebook de McKenna Group

```

12. Actualizaciones operativas (2026-04-23)

Esta sección documenta cambios recientes aplicados en producción para mejorar confiabilidad de preventa/postventa MeLi y supervisión autónoma.

12.1 Preventa (`resp ...`) — cierre correcto de pendientes

- Archivo: `app/routes.py`, `app/services/meli_preventa.py`
- Antes: una pregunta podía marcarse como `respondida=true` incluso cuando el POST a `https://api.mercadolibre.com/answers` fallaba.
- Ahora:
 - Se marca `respondida=true` `**solo**` cuando MeLi confirma éxito (`200` o `201`).
 - Si falla el envío, la pregunta queda pendiente para reintento.
 - El caso de entrenamiento (`casos_preventa.json`) se guarda solo en éxito.

Impacto: se elimina falso positivo "respondida" sin haber respondido al cliente final.

12.2 Ambigüedad de comandos cortos (`resp 415:`)

- Archivo: `app/routes.py`
- Mejora:
 - Si el sufijo corto coincide con varias preguntas, el sistema no adivina.
 - Envía alerta de "código ambiguo" y exige formato completo:
 - `resp preventiva <question_id>: tu respuesta`

Impacto: evita respuestas cruzadas a preguntas equivocadas.

12.3 Integración OpenHands/Ollama — fallback operativo

- Archivo: `app/services/local_ai.py`
- Mejora:
 - Si `OPENHANDS_CMD` no está definido, usa por defecto:
 - `python3 scripts/openhands_bridge.py --payload {payload}`
 - Detección robusta de modelos Ollama instalados (`ollama list`).

- ``modelo_efectivo``:
- usa ``LOCAL_AI_MODEL`` si existe;
- si no, toma el primer modelo local disponible.

Impacto: autocorrector deja de depender de configuración manual mínima para funcionar.

12.4 Supervisor de colas MeLi (preventa + postventa)

- Archivo: ``app/monitor.py``
- Nuevo hilo:
- ``_supervisar_colas_meli()``
- Función:
- Detecta pendientes estancados.
- Envía alertas accionables al grupo correcto:
- Preventa: ``resp ...` / `resp preventa ...``
- Postventa: ``posventa ...``
- Envía resumen al grupo de alertas sistemas.

Variables:

```
SUPERVISOR_COLAS_INTERVALO_SEG=600
SUPERVISOR_PREVENTA_UMBRAL_MIN=35
SUPERVISOR_POSTVENTA_UMBRAL_MIN=30
SUPERVISOR_PREVENTA_COOLDOWN_MIN=30
SUPERVISOR_POSTVENTA_COOLDOWN_MIN=20
SUPERVISOR_RESUMEN_COOLDOWN_MIN=60
```

12.5 Anti-ruído + reconciliación con estado real de MeLi

- Archivo: ``app/monitor.py``
- Problema resuelto:
- Supervisor reenviaba alertas de casos ya respondidos fuera del flujo local.
- Corrección:
- **Preventa:** consulta ``GET /questions/{id}`` antes de alertar.
- Si ya no está en ``UNANSWERED``, auto-marca ``respondida=true``.
- **Postventa:** consulta ``GET /messages/packs/{pack_id}/sellers/{seller_id}?tag=post_sale``.
- Si ``msg_id`` pendiente ya no existe, o vendedor respondió después, elimina pendiente local.
- Cooldown por canal (preventa/postventa/resumen) para evitar spam repetitivo.

Impacto: alertas mucho más fieles al estado real de Mercado Libre.

12.6 Reinicio requerido tras cambios

Cada vez que se modifiquen ``monitor.py``, ``routes.py``, ``local_ai.py`` o variables ``env``, reiniciar:

```
sudo systemctl restart agente-pro
sudo systemctl is-active agente-pro
```

Esperado: ``active``.